# CMSC 313 Project 2

Name: _____

Assigned: 10/6/2026

Due: 10/27/2026 at 11:59pm

## Project Description

In this project, you will design several components of a 4-bit multi-purpose processor using Logisim Evolution. A skeleton file named Project2_Skeleton.circ will help you get started – make sure to download it from Blackboard. Read these directions **carefully** before starting.

Please rename the skeleton file to Project2_YOURLASTNAME.circ before submitting the project to Blackboard. Do **NOT** change any of the subcircuit names in Logisim Evolution. Do **NOT** add, remove, or modify the properties of any input or output pins. If you move any input or output pins, you must keep them in the **same vertical ordering**. If you do not follow these instructions, you may have up to 50% of your project grade deducted.

Update Logisim to version 5.0.0, if you are not currently on that version:

- https://github.com/logisim-evolution/logisim-evolution/releases

## Given Sub-Circuits

You have been given several working sub-circuits in the Skeleton file. Do **NOT** change any of these sub-circuits.

**Incrementor** – An Incrementor component.

- Incrementor inputs:
  - $A$ – A 4-bit integer
- Incrementor outputs:
  - $Q$ – A 4-bit integer that is equal to $A_3A_2A_1A_0$plus 1. If $A_3A_2A_1A_0=1111$, the incrementor overflows and $Q_3Q_2Q_1Q_0=0000$.

**Arithmetic_Logic_Unit** – An ALU that is behaviorally identical to Project 1.

- ALU Inputs:
  - $A$ – A 4-bit data input
  - $B$– A second 4-bit data input
  - $Opcode$ – A three-bit input $(O_2O_1O_0)$ that determines the ALU's operation.
- ALU Outputs:
  - $Q$ – The 4-bit output from the ALU.

- $Z_{flag}$ – The Zero flag is 1 when $Q_3 Q_2 Q_1 Q_0 = 0000$, and 0 otherwise.

**ALU Operations:**

| Opcode | Operation |
|---|---|
| 000 | $Q = B$ (No_Operation / Move) |
| 001 | $Q = A = ¿ B$ (Equality_Comparison) |
| 010 | $Q = A$ XOR $B$ (Bitwise_XOR) |
| 011 | $Q = \overline{A}$ (Inverter) |
| 100 | $Q = A + 1$ (Increment) |
| 101 | $Q = A + B$(Addition) |
| 110 | $Q = A - B$ (Subtraction) |
| 111 | $Q = A * B$ (Multiplication) |

**Register_File** – A file of four general-purpose registers that is almost behaviorally identical to HW 3.

- Register_File inputs:
  - $D$ – the 4-bit data input.
  - $WE$ – the Write-Enable input. If it is 1, then exactly 1 of the registers should be write-enabled. If it is 0, then none of the registers should be write-enabled.
  - $W_{id}$ – the 2-bit ID of the register that should be write-enabled, if $WE$ is 1.
  - $R_{¿}$ – the 2-bit ID of a register. The value of the register with ID $R_{¿}$ should be stored in $Q_A$.
  - $R_{¿}$ – the 2-bit ID of another register. The value of the register with ID $R_{¿}$ should be stored in $Q_B$.
  - $Clk$ is the clock input.
  - $Reset$ is the reset input. If $Reset = 1$, all registers become 0.
- Register_File outputs:
  - $Q_A$ – the value stored in the register with the ID $R_{¿}$.
  - $Q_B$ – the value stored in the register with the ID $R_{¿}$.

**Flag_Register** – Normally, an ALU has flag outputs describing the result of its last operation (such as if the result was 0, if it was negative, if an overflow happened, etc). These flags are stored together in a single register. For simplicity, the flag register will only store one flag, named $Z_{Flag}$.

- Flag_Register inputs:
  - $Z_{¿}$ – the zero flag to store in the flag register. Its value is 1 if the ALU's output was equal to zero, and its value is 0 if the ALU's output was non-zero.
  - $WE$ – the write-enable bit. Instructions are either ALU instructions or Jump instructions. All ALU instructions modify one of the registers in the register file, so $WE = 1$ for these instructions. Jump instructions do not affect the register file, so $WE = 0$ for them. If $WE$ is 0, the flag register does not update.

    - $Clk$ – the clock input.
    - $Reset$ – the reset input. If $Reset = 1$, the flag register becomes 0.
- Flag_Register outputs:
    - $Z_{flag}$: The value of the zero flag currently stored in the flag register.

## Part 1: PC_Register (25 pts)

The Program Counter (PC) register is a component that stores the address of the instruction to execute next. The ROM component in the Multi_Purpose_Processor has 16 9-bit memory entries. The first memory entry has the address 0, the second memory entry has the address 1, and so on. (The sixteenth memory entry has the address 0xF, since the built-in Logisim ROM and registers show their values in hexadecimal).

- PC_Register inputs:
    - $Do_{Jump}$– If this bit is 1, the Program Counter's register should become equal to $Jump_{Addr}$. If this bit is 0, the Program Counter's register should increment instead.
    - $Jump_{Addr}$ – The 4-bit address in the Multi_Purpose_Processor's ROM component to "jump to". If $Do_{Jump}$ is 1, the program counter's register should become equal to $Jump_{Addr}$.
    - $Clk$ – The clock cycle input.
    - $Reset$ – When this input is 1, the Program Counter's register should become 0.
- PC_Register outputs:
    - $M_{addr}$ – The 4-bit address of the machine code (in ROM) that should be executed next by the Multi_Purpose_Processor.

When the program begins, the PC register should be initialized to 0, meaning that the instruction at address 0 will be executed first. At the rising edge of each clock cycle, if the $Do_{Jump}$ input is 0, the Program Counter's register should increase by 1 using the given Incrementor component. If the $Do_{Jump}$ input is 1, the Program Counter's register should become equal to the $J_{addr}$ input, rather than incrementing.

In the skeleton file, a Multiplexor, an Incrementor, and a Logisim register have been placed at the bottom of the circuit. They will each be used somewhere in PC_Register.

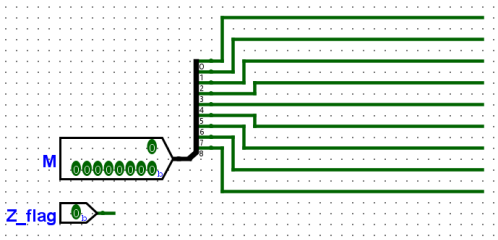
## Part 2: Control_Unit (40 pts)

The Control Unit is responsible for interpreting machine code during the "Decode" phase of the Fetch/Decode/Execute cycle. Each machine code instruction is a 9-bit value, represented as 3-digit hexadecimal values in the Multi_Purpose_Processor's ROM component. For example, the hexadecimal

value 0x1A7 is equivalent to the 9-bit machine code 1 1010 0111.

- The inputs to the Control Unit are:
  - $M$ – The 9-bit machine code instruction.
  - $Z_{flag}$ – The value of the zero flag.

A splitter has been given to you, to separate $M$ into its 9 bits ($M_8 M_7 M_6 M_5 M_4 M_3 M_2 M_1 M_0$). $M_0$ is the wire at the top and $M_8$ is the wire at the bottom.




$M$ for non-Jump instructions:

| $M_8$ | $M_7$ | $M_6 M_5 M_4$ | $M_3 M_2$ | $M_1 M_0$ |
|---|---|---|---|---|
| Jump_inst r | Operand | Opcode | Dst ID | Src ID |

$M$ for Jump instructions:

| $M_8$ | $M_7$ | $M_6 M_5 M_4$ | $M_3 M_2 M_1 M_0$ |
|---|---|---|---|
| Jump_inst r | Operand | Opcode | Jump_Addr |

- Control Unit outputs:
  - $WE$ – the write-enable bit. Instructions are either ALU instructions or Jump instructions. All ALU instructions modify one of the registers in the register file, so $WE = 1$ for these instructions. Jump instructions do not affect the register file, so $WE = 0$ for them. If $WE$ is 0, the flag register does not update.
  - $W_{id}$ – the 2-bit ID of the register that should be write-enabled, if $WE$ is 1.
  - $R_i$ – the 2-bit ID of a register. The value of the register with ID $R_i$ should be stored in $Q_A$.
  - $R_i$ – the 2-bit ID of another register. The value of the register with ID $R_i$ should be stored in $Q_B$.
  - $Opcode$ – The 3-bit opcode indicating which operation the ALU should perform.

- $Operand$ – If this value is 1, a multiplexor in the Multi_Purpose_Processor will forward $I$ to the B input of the ALU. If it is 0, it will forward $Q_B$ from the register file to the B input of the ALU.
- $Do_{Jump}$ – Whether or not the current instruction should perform a jump. This depends on whether the current instruction is a jump instruction, the opcode, and possibly the state of $Z_{¿}$ flag.
- $Jump_{Addr}$ – The 4-bit address to jump to.

## Instruction Set Architecture

Below is a table of the Multi_Purpose_Processor's Instruction Set Architecture

- It shows the upper 5 bits of $M$, the assembly instruction, the instruction's behavior, and whether or not the instruction affects the Z_flag.
- $dst$ denotes the destination register ($W_{id}$ and $R_{¿}$) and $src$ denotes the source register ($R_{¿}$).
- The symbol ":=" is the assignment operator.

| Jump_instr ($M_8$) | Operand ($M_7$) | Opcode ($M_6 M_5 M_4$) | Assembly Instruction | Instruction Behavior | Z_Flag Affected? |
|---|---|---|---|---|---|
| 0 | 0 | 000 | MOV dst, src | dst := src | Yes |
| 0 | 0 | 001 | TEST dst, src | dst := dst == src | Yes |
| 0 | 0 | 010 | XOR dst, src | dst := dst ^ src | Yes |
| 0 | 0 | 011 | NOT dst | dst := ~dst | Yes |
| 0 | 0 | 100 | INC dst | dst := dst + 1 | Yes |
| 0 | 0 | 101 | ADD dst, src | dst := dst + src | Yes |
| 0 | 0 | 110 | SUB dst, src | dst := dst – src | Yes |
| 0 | 0 | 111 | MUL dst, src | dst := dst * src | Yes |
| 0 | 1 | 000 | MOV dst, I | dst := I | Yes |
| 0 | 1 | 001 | TEST dst, I | dst := dst == I | Yes |
| 0 | 1 | 010 | XOR dst, I | dst := dst ^ I | Yes |
| 0 | 1 | 011 | NOT dst | dst := ~dst | Yes |
| 0 | 1 | 100 | INC dst | dst := dst + 1 | Yes |
| 0 | 1 | 101 | ADD dst, I | dst := dst + I | Yes |
| 0 | 1 | 110 | SUB dst, I | dst := dst – I | Yes |
| 0 | 1 | 111 | MUL dst, I | dst := dst * I | Yes |
| 1 | Either | 000 | JZ J_addr | PC := $Jump_{Addr}$ if $Z_{flag}$ == 1 | No |
| 1 | Either | 001 | JNZ J_addr | PC := $Jump_{Addr}$ if | No |

| | | | | | |
|---|---|---|---|---|---|
| | | | | $Z_{flag}$ == 0 | |
| 1 | Either | 010 | JMP J_addr | PC := $Jump_{Addr}$ | No |
| 1 | Either | 011 | Not Used | N/A | N/A |
| 1 | Either | 100 | Not Used | N/A | N/A |
| 1 | Either | 101 | Not Used | N/A | N/A |
| 1 | Either | 110 | Not Used | N/A | N/A |
| 1 | Either | 111 | Not Used | N/A | N/A |

## Instructions for Wiring the Control Unit

You may use any basic logic gates (AND, OR, NOT, etc.) and splitters in the Control Unit. You may not use any other components. You may configure your basic logic gates so that they have more than 2 inputs.

The lower 4 bits of $M$ are used for two different purposes. If the current instruction is a Jump instruction, then $M_3 M_2 M_1 M_0$ is the 4-bit address that the program intends to jump to. If the current instruction is not a Jump instruction, then $M_3 M_2$ is used for $W_{id}$and $R_{¿}$, and $M_1 M_0$ is used for $R_{¿}$.

- $WE$ – This should be 0 if the current instruction is a Jump instruction, and 1 if the current instruction is not a Jump instruction. The $M_8$ bit of the machine code determines whether or not the current instruction is a Jump instruction or not.
- $W_{id}$ – This two-bit value should be wired directly to $M_3 M_2$. Make sure that the upper bit of $W_{id}$ connects to $M_3$ and the lower bit of $W_{id}$ connects to $M_2$.
- $R_{¿}$ – This two-bit value should be wired to $M_3 M_2$.
- $R_{¿}$ – This two-bit value should be wired to $M_1 M_0$.
- $Opcode$ – This 3-bit value should be wired to $M_6 M_5 M_4$.
- $Operand$ – This 1-bit value should be wired to $M_7$.
- $Do_{Jump}$ – This 1-bit value indicates whether a Jump should be taken. More directions below.
- $Jump_{Addr}$ – This 4-bit value should be wired to $M_3 M_2 M_1 M_0$.

The $Do_{Jump}$ output is a bit more complicated than the rest. It depends on several things. First, $Do_{Jump}$ can only be 1 if the instruction is a jump instruction ($M_8$ = 1). It also depends on the type of Jump instruction, and potentially the value of $Z_{flag}$.

- JZ – If $M_8 is 1$, $Opcode\ (M ¿¿ 6\ M_5 M_4) ¿$ is 000 and $Z_{flag}$ is 1, then $Do_{Jump}$should be 1.
- JNZ - If $M_8 is 1$, $Opcode\ (M ¿¿ 6\ M_5 M_4) ¿$ is 001 and $Z_{flag}$ is 0, then $Do_{Jump}$should be 1.
- JMP - If $M_8 is 1$, $Opcode\ (M ¿¿ 6\ M_5 M_4) ¿$ is 010, then $Do_{Jump}$should be 1.
- $Do_{Jump}$ should be 0 for all other values of $M$.

## Machine Code Example

Here is an example of an assembly instruction and its machine code representation:

- Assembly: ADD Reg_00, Reg_11
- Machine code: 0 0101 0011
- Operation: Reg_00 := Reg_00 + Reg_11

The 9 bit machine code $M$ from the example is broken down into the following parts:

- $M_1 M_0$ depend on whether this is an ALU instruction or a Jump instruction.
    - When $Jump_{instr}$ == 0, $M_1 M_0$ become $R_{ii}$. In this example, this means to read from the register with address 11.
    - When $Jump_{instr}$ == 1, $M_1 M_0$ are the lower two bits of $Jump_{Addr}$, the 4-bit address to jump to.
- $M_3 M_2$ also depend on whether this is an ALU instruction or a Jump instruction.
    - When $Jump_{instr}$ == 0, $M_3 M_2$ become $W_{id}$ and $R_{ii}$. In this example, this means to read from the register with address 00 and store the result in the register with address 00.
    - When $Jump_{instr}$ == 1, $M_3 M_2$ are the higher two bits of $Jump_{Addr}$, the 4-bit address to jump to.
- $M_6 M_5 M_4$ are the opcode for the ALU. The opcode controls which operation the ALU executes. In this example, $M_6 M_5 M_4$ are 101, which means that the ALU performs addition.
- $M_7$ is the $Operand$ bit. Since it is 0, the $B$ input of the ALU comes from $Q_B$, rather than $I$.
- $M_8$ is the $Jump_{instr}$ bit. Since it is 0, this is not a Jump instruction.

## More Machine Code Examples

Example 1:

- Assembly: ADD Reg_01, I
- Machine code: 0 1101 01XX
- Behavior: Add the input $I$ to Reg_01, and store the result in Reg_01. $M_1 M_0$ does not matter in this example, and can have any value.

Example 2:

- Assembly: INC Reg_10
- Machine code: 0 X100 10XX
- Increment the contents of Reg_10 by 1 and store the result in Reg_10. $M_7$ and $M_1 M_0$ do not matter in this example, and can have any value.

Example 3:

- Assembly: JZ 6
- 1 X000 0110

- Jump to the machine code at address 6 if $Z_{flag}$is equal to 1. $M_7$ does not matter in this example. $M_3 M_2 M_1 M_0$ (which are normally used as register IDs in ALU instructions) are equal to 6, which is the address that will be jumped to.

## Part 3: Multi_Purpose_Processor (35 pts)

- Multi_Purpose_Processor inputs:
  - $I$ – a 4-bit input to the ALU.
  - $Clk$ – a Logisim clock component.
  - $Reset$ – the reset input. If $Reset = 1$, all registers become 0.
- The outputs of the multi-purpose processor are:
  - $Q_A$ – a 4-bit value from the read port of the register file.
  - $Q_B$ – another 4-bit value from the read port of the register file.

A multiplexor and a ROM component have been placed at the bottom of the Multi_Purpose_Processor. You should use them somewhere in the Multi_Purpose_Processor. Place an Arithmetic_Logic_Unit, Register_File, Flag_Register, PC_Register, and Control_Unit into the Multi_Purpose_Processor. Then, wire together a working Multi_Purpose_Processor.

The ALU's $A$ input should receive the $Q_A$ output of the Register_File. The ALU's $B$ input should receive either the Register_File's $Q_B$ output, or $I$, depending on the value of $Operand$.

You may not use logic gates, splitters, or any other components in Logisim to complete the Multi_Purpose_Processor, except for wires and the components listed above.

## PC_Register Testing

In the top bar in Logisim-Evolution, click Simulate -> Reset Simulation. Then, navigate to the PC_Register component.

1. Click the Reset input twice. The value of the register should be 0, and the value of $M_{Addr}$ should be 0000.
2. Click the Clk input eight times. Each time Clk is 1, the register value and $M_{Addr}$ should increase by 1. After doing this, the value of the register should be 4, and $M_{addr}$should be 0100.
3. Set $Do_{Jump}$ to 1 and set $Jump_{Addr}$ to 1101. Then, click Clk twice. The value of the register should be "d" (13 in hexadecimal) and $M_{addr}$ should be 1101.

## Control Unit Testing

In the top bar in Logisim-Evolution, click Simulate -> Reset Simulation. Navigate to the Control_Unit.

1. Select the $M$ input pin. On the bottom left window, scroll down to "Radix" and set it to Binary. Set the following values:
    a. $M$ = 1 0000 1000
    b. $Z_{flag}$ = 1
2. After step 1, the output pins of the Control Unit should be:
    a. $WE = 0$
    b. $W_{id} = 10$
    c. $R_{¿}$ = 10
    d. $R_{¿}$ = 00
    e. $Opcode$ = 000
    f. $Operand$ = 0
    g. $Do_{Jump}$ = 1
    h. $Jump_{Addr}$ = 1000
3. Next, set these input values:
    a. $M$ = 0 1101 1110
    b. $Z_{flag}$ = 0
4. After step 3, the output pins of the control unit should be:
    a. $WE = 1$
    b. $W_{id} = 11$
    c. $R_{¿}$ = 11
    d. $R_{¿}$ = 10
    e. $Opcode$ = 101
    f. $Operand$ = 1
    g. $Do_{Jump}$ = 0
    h. $Jump_{Addr}$ = 1110

## Multi_Purpose_Processor Testing

In the top bar in Logisim-Evolution, click Simulate -> Reset Simulation. Navigate to the Multi_Purpose_Processor, then follow these instructions.

1. Right-click the ROM component and select "Edit Contents". Enter the following:
    a. 080 040 004 0E4 116 088 126 (leaving the rest of the entries on the right as 000)
2. Set the value of $I$ to 0011.
3. Repeatedly click the $Clk$ twice to simulate a clock cycle. After each clock cycle, check the results of your program below. If your Multi_Purpose_Processor is wired correctly, you should see these results:

a. After the first clock cycle, $Q_A$and $Q_B$ are 0011
b. After the second clock cycle, $Q_A$ is 0000 and $Q_B$ is 0100
c. After the third clock cycle, $Q_A$and $Q_B$ are 0100
d. After the fourth clock cycle, $Q_A$is 0001 and Q_B is 0000
e. After the fifth clock cycle, the value 126 is highlighted in the ROM, $Q_A$is 0001 and Q_B is 0000.
f. Cycling the clock after that does nothing.

These tests are not comprehensive, and you are encouraged to test each component more thoroughly.

## Grading Breakdown

- 25 points: Program Counter works correctly.
- 40 points: Control Unit works correctly.
- 35 points: Multi_Purpose_Processor works correctly.

Partial credit will be given for circuits whose designs are nearly correct but have minor errors.

## How to Submit

Test your circuit to make sure that everything works properly. Make sure your file is named Project2_YOURLASTNAME.circ. Submit your completed .circ file on Blackboard.